3.4.2 Astra AI project: Integrating Gemini vision queries

In this section, I will guide you through the technical steps to integrate Gemini vision capabilities into your Astra AI project. This involves updating the UI to handle multimedia prompts and using Gemini models for processing images alongside text inputs.

TECHNICAL STEPS TO INTEGRATION

Here's what we're going to build:

- A system that can process multimedia prompts (text + images)
- An updated UI that allows users to upload images alongside text queries
- A backend that uses Gemini vision capabilities to analyze images and provide insightful responses

NOTE The source code for the following example is available at https://github.com/Generative-AI-Web-Apps/Code/tree/main/ch03/chat-multimedia.

Let's start with the backend implementation changes. To handle image data on the backend, you'll need to modify your message-processing logic. Instead of just accepting a text message, the backend should be prepared to receive a message object that includes both text and an image URL. The `processIncomingMessages` function in your route.js file can be updated to check for an `imageUrl` field in the request. If present, it will reformat the last message in the conversation to be an array of "parts." This array includes an object with `type: "text"` for the user's prompt and an object with `type: "image"` containing the image URL. This new, enriched message object is then passed to the `streamText` function.

Listing 3.14 `processIncomingMessages` function in route.js

```
async function processRequestMessages(req) {
  const { messages, data } = await req.json();
  if (!data?.imageUrl) return messages;

  const initialMessages = messages.slice(0, -1);
  const lastMessage = messages[messages.length - 1];

  return [
    ...initialMessages,
    {
      ... lastMessage,
      content: [
        { type: "text", text: lastMessage.content },
        {
          type: "image",
          image: data.imageUrl,
        },
      ],
    },
  ];
};
```

Parses the incoming request to extract messages and additional data

If there's no image URL provided in the data, returns the original messages

Gets the last message

Modifies the last message so that it includes both its original text content and the additional image content. This ensures that the final message in the conversation contains the multimedia prompt.